

ЛАБОРАТОРНА РОБОТА №4.

ІНТЕГРАЦІЯ ФРОНТЕНДУ З БЕКЕНДОМ

Мета роботи: Сформувані практичні навички інтеграції фронтенд-частини вебзастосунку з бекенд-API: виконувати HTTP-запити через `fetch()`, коректно відображати отримані дані та стани інтерфейсу (`loading/success/empty/error`), обробляти помилки мережі й відповіді з кодами 4xx/5xx, налаштовувати та перевіряти CORS для роботи у браузері, а також узгоджувати контракти обміну даними (DTO) з дотриманням принципу сумісності форматів («не ламати» структуру відповідей під час змін).

Теоретичні відомості

Підготовка середовища інтеграції

Чому фронтенд і бекенд у dev працюють окремо

У навчальному проєкті фронтенд і бекенд – два окремі процеси:

- бекенд слухає свій порт (наприклад, `http://localhost:3000`) і віддає JSON через API;
- фронтенд відкривається у браузері з іншого порту (наприклад, `http://localhost:5500`) і робить запити до бекенду через `fetch()`.

Це нормально і типово для розробки. Однак для браузера це означає, що сторінка має свій `origin`, а API – інший `origin`.

Origin (спрощено) складається з трьох частин:

1. схема (`http` або `https`)
2. хост (`localhost`, `127.0.0.1`, домен)
3. порт (`5500`, `3000`, `5173`)

Приклад:

`http://localhost:5500` $\neq$ `http://localhost:3000` (порти різні $\rightarrow$ origin різні)

Саме через це у лабораторній роботі №4 з'являється тема CORS (буде детально розглянуто трохи далі).

Чому не можна відкривати фронтенд як file://...

Якщо відкрити index.html подвійним кліком, у адресному рядку буде щось на кшталт:

file:///C:/.../index.html

Для браузера це «особливий» режим:

- Origin може бути null або поводитись нестандартно;
- CORS-правила у такому режимі можуть працювати непередбачувано;
- частина можливостей (модулі, імпорти, fetch до локальних ресурсів) може бути обмежена.

Вимога для цієї лабораторної роботи: фронтенд має запускатися через локальний веб-сервер (http://...), навіть якщо це «просто HTML+JS».

Як правильно запускати фронтенд без фреймворка

Найпростіші варіанти. Можна використовувати будь-який один.

Варіант А: VS Code Live Server (рекомендовано)

1. Встановити розширення Live Server у VS Code.
2. Відкрити папку фронтенду.
3. Відкрити index.html → натиснути «Go Live».

Буде отримано URL на кшталт:

http://127.0.0.1:5500/ або http://localhost:5500/

Варіант В: http-server через Node

У папці фронтенду:

```
npx http-server -p 5500
```

Потім відкрити:

```
http://localhost:5500
```

Варіант С: Python (якщо встановлений)

```
python -m http.server 5500
```

Це не єдині способи це зробити, але самі прості.

Домовленість про порти і адреси

Щоб не було хаосу, варто зафіксувати типову схему:

- Frontend: http://localhost:5500
- Backend: http://localhost:3000

BASE_URL для API: одна точка правди

Погана практика: писати URL бекенду по всьому коду, типу:

```
fetch("http://localhost:3000/api/v1/posts")
```

десятки разів.

Правильніше: винести базову адресу в константу. Наприклад, у файлі config.js:

```
export const API_BASE_URL = "http://localhost:3000/api/v1";
```

Далі в коді:

```
import { API_BASE_URL } from "./config.js";
fetch(`${API_BASE_URL}/posts`);
```

Це:

- спрощує зміну порту/адреси;
- зменшує кількість «магічних рядків»;
- зручно для перевірки викладачем.

У наступних розділах буде побудовано apiClient, де API_BASE_URL стане основою.

Мінімальна структура фронтенду під інтеграцію

Навіть без фреймворків бажано мати мінімальну структурування, щоб не змішувати все в одному файлі.

Рекомендована структура (дуже проста):

```
frontend/
  index.html
  src/
    config.js      // API_BASE_URL
    apiClient.js   // функції для запитів (ЛР4)
    ui.js          // рендеринг/DOM-операції (ЛР1)
    main.js        // точка входу: викликає apiClient і ui
```

Це не «архітектура заради архітектури», а практичне розділення:

- apiClient відповідає за HTTP і помилки,
- ui – за DOM,
- main – за сценарії (завантажити список, показати деталі, обробити submit форми тощо).

fetch() у браузері: реальна поведінка і типові пастки

У лабораторній роботі №4 fetch() – це основний інструмент, через який фронтенд спілкується з бекендом. Важливо розуміти реальну поведінку fetch, бо багато типових помилок виникає саме через неправильні очікування.

fetch і помилки: 404/500 – це не exception

Найчастіша пастка: студент пише

```
try {
  const res = await fetch(url);
  const data = await res.json();
} catch (e) {
  // тут очікують 404/500
}
```

Але catch спрацьовує не на 404/500, а переважно лише на:

- помилки мережі (сервер недоступний, DNS, розрив з'єднання),
- CORS-блокування (браузер не дає читати відповідь),
- інколи – «биті» низькорівневі ситуації.

404/500 – це «успішний» мережевий запит, просто відповідь має status = 404/500. Тобто треба перевіряти response.ok або response.status.

Базовий шаблон «коректного» fetch

Мінімальний шаблон для GET:

```
async function getJson(url) {
  const response = await fetch(url);

  // 1) Перевіряємо HTTP-статус
  if (!response.ok) {
    // 2) Намагаємось прочитати тіло помилки
    const text = await response.text(); // безпечніше за json()
    throw new Error(`HTTP ${response.status}: ${text}`);
  }

  // 3) Якщо все добре – читаємо JSON
  return await response.json();
}
```

Чому в помилці читається `text()`, а не `json()`?

- бекенд може повернути HTML/текст (наприклад, дефолтний error page),
- студент може випадково звернутися не до API, а до іншого маршруту,
- навіть якщо бекенд повертає JSON-помилку, `text()` все одно працює завжди.

Пізніше, коли буде уніфікований формат помилок, можна «розумно» парсити JSON – але базовий підхід має бути надійним.

Відправка JSON: POST/PUT/PATCH і типові нюанси

Найчастіше студентам треба створювати або редагувати сутність. Для цього відправляють JSON:

```
async function postJson(url, bodyObj) {
  const response = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(bodyObj)
  });

  if (!response.ok) {
    const text = await response.text();
    throw new Error(`HTTP ${response.status}: ${text}`);
  }

  // У різних API може бути 201 з JSON або 204 без тіла
  if (response.status === 204) return null;

  return await response.json();
}
```

Типові помилки:

- Забування `JSON.stringify` → сервер отримує `[object Object]` або порожнє тіло.
- Не поставлення `Content-Type: application/json` → бекенд не розбирає тіло.
- Очікування JSON там, де сервер повернув 204 No Content → `response.json()` падає.

Preflight (OPTIONS) як наслідок Content-Type: application/json

Важливе практичне спостереження:

- GET часто проходить без preflight.
- POST з Content-Type: application/json майже завжди викликає preflight-запит OPTIONS.

Тому ситуація «GET працює, а POST – ні» часто означає:

- на бекенді не налаштований CORS для OPTIONS,
- не дозволені заголовки,
- або не дозволений метод.

Це не проблема fetch, це проблема CORS-конфігурації.

Обробка «помилки мережі» та CORS: як відрізнити від 4xx/5xx

З клієнтського боку виглядає так:

- якщо сервер відповів 404 – у результаті response.status = 404;
- якщо сервер недоступний або CORS блокує – виконується catch, часто з повідомленням типу: «TypeError: Failed to fetch»

Це повідомлення не говорить прямо, що саме сталося. Тому правило:

1. Дивимось DevTools → Network:
2. чи є запит?
3. який статус?
4. чи був preflight OPTIONS?
5. Дивимось Console:
 - CORS-повідомлення зазвичай дуже прямолінійне (що саме не дозволено).

Скасування запитів і таймаут через AbortController

У базовій версії fetch не має «вбудованого» таймауту. Якщо бекенд завис або мережа проблемна, запит може «висіти». Для підвищення надійності використовують AbortController.

Приклад: таймаут 10 секунд

```
async function fetchWithTimeout(url, options = {}, timeoutMs = 10_000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
```

```

    ...options,
    signal: controller.signal
  });
  return response;
} finally {
  clearTimeout(id);
}
}

```

Використання:

```
const response = await fetchWithTimeout(`${API_BASE_URL}/posts`, {}, 10_000);
```

У catch можна перевірити, чи це abort:

```

catch (e) {
  if (e.name === "AbortError") {
    // показати: "Запит перевищив таймаут"
  }
}

```

«Гонки» (race conditions): коротко, але корисно

Сценарій: користувач швидко відкриває «Деталі» різних записів. Запит №1 стартував першим, але прийшов пізніше за запит №2. В результаті UI показує «старі» дані.

Мінімальна профілактика без складної архітектури:

- або скасовувати попередній запит через AbortController,
- або зберігати «номер запиту» і оновлювати UI лише якщо відповідь актуальна.

Для цього курсу достатньо усвідомлення проблеми; але самостійна реалізація буде бонусом.

Клієнтський «API-шар»: як не розмазати fetch по всьому UI

На практиці інтеграція «ламається» не тому, що fetch() складний, а тому що його швидко починають копіювати в різні місця: в обробники кнопок, у форми, у завантаження сторінок. У результаті:

- десь перевірили response.ok, десь забули;
- десь обробили 204, десь отримали помилку Unexpected end of JSON input;
- десь показали текст помилки, десь просто console.log;
- URL бекенду розкиданий по коду.

Тому доцільно зробити мінімальний `apiClient` – 1 файл, де зосереджена логіка HTTP-викликів і нормальна помилка для UI.

Ідея: один модуль, який «вміє» говорити з API

Мінімальний підхід:

- `config.js` – містить `API_BASE_URL`
- `apiClient.js` – містить:
 - одну «внутрішню» функцію `request()`, яка робить `fetch`, перевіряє статус, парсить JSON або повертає текст помилки;
 - набір «зовнішніх» функцій під ваші ендпоінти: `getList`, `getById`, `create`, `update`, `remove`.

Таким чином у UI коді не буде деталей протоколу – лише «бізнес-дії» типу «завантаж список».

Узгоджений формат помилки на фронтенді

Бекенд може повертати різний формат помилки (`ProblemDetails`, власний `error DTO`, або просто текст). Щоб UI не гадав, що саме прийшло, зручно на клієнті привести все до одного вигляду.

Приклад того, як може виглядати помилка в UI:

```
{
  status: 400,
  message: "Validation error",
  details: "...",
  errors: { fieldName: ["..."] } // опційно
}
```

Наразі достатньо спрощеного варіанту:

- `status`
- `message`
- `details` (опційно)

Базова реалізація `request()` (JS, без фреймворка)

Файл `src/apiClient.js`:

```
import { API_BASE_URL } from "../config.js";

async function request(path, options = {}) {
  const url = `${API_BASE_URL}${path}`;
```

```

let response;
try {
  response = await fetch(url, options);
} catch (e) {
  // мережа / CORS / бекенд вимкнений
  throw {
    status: 0,
    message: "Помилка мережі або CORS",
    details: e?.message || String(e),
  };
}

// 204: немає тіла
if (response.status === 204) {
  if (!response.ok) {
    throw { status: response.status, message: "Помилка", details: "No Content" };
  }
  return null;
}

// Прагнемо бути «стійкими»: спочатку читаємо як text
const rawText = await response.text();

// Успіх
if (response.ok) {
  // Якщо тіло пусте – повертаємо null
  if (!rawText) return null;

  // Спробуємо JSON, якщо ні – повернемо текст (на випадок простих API)
  try {
    return JSON.parse(rawText);
  } catch {
    return rawText;
  }
}

// Неуспіх: намагаємося витягнути зрозумілу помилку
let errPayload = null;
try {
  errPayload = rawText ? JSON.parse(rawText) : null;
} catch {
  // ок, лишимо rawText
}

throw {
  status: response.status,
  message: errPayload?.title || errPayload?.message || "HTTP помилка",
  details: errPayload?.detail || rawText || `HTTP ${response.status}`,
  errors: errPayload?.errors || null,
};
}

```

Пояснення ключових рішень (коротко):

- try/catch лише навколо fetch, щоб ловити мережеві та CORS-проблеми.
- response.text(), щоб прочитати відповідь один раз і далі парсити JSON – так безпечніше.
- статус 204 оброблено окремо.
- throw кидає об'єкт помилки, зручний для UI.

У JS можна кидати не тільки Error. Для навчального проекту це допустимо. Можна кидати new Error(), але тоді важче передавати status/errors.

Функції під конкретну сутність (заготовки)

Якщо основна сутність – <main-entity>:

```
export async function getList() {
  return await request("/<main-entity>", { method: "GET" });
}

export async function getById(id) {
  return await request(`/<main-entity>/${encodeURIComponent(id)}`, { method: "GET" });
}

export async function create(dto) {
  return await request("/<main-entity>", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dto),
  });
}

export async function update(id, dto) {
  return await request(`/<main-entity>/${encodeURIComponent(id)}`, {
    method: "PUT", // або PATCH – залежить від вашого бекенду
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dto),
  });
}

export async function remove(id) {
  return await request(`/<main-entity>/${encodeURIComponent(id)}`, {
    method: "DELETE",
  });
}
```

Це не «готове рішення», а шаблон.

Використання apiClient у UI: мінімальний приклад інтеграції

У src/main.js (приклад для завантаження списку):

```
import { getList } from "../apiClient.js";
```

```

async function load() {
  try {
    // 1) показати loading у UI (див. розділ 4)
    const items = await getList();

    // 2) відрендерити items
    console.log(items);
  } catch (err) {
    // err має status/message/details
    console.error(err);
    // показати повідомлення користувачу
  }
}

load();

```

UI тепер не знає, як парсити JSON і що робити з 404/500 – цим займається `apiClient`.

DTO → ViewModel: коли варто робити перетворення

Іноді DTO з бекенду не дуже зручний для UI:

- дати в ISO-форматі, а треба «людсько»;
- вкладені об'єкти, а UI показує тільки пару полів;
- назви полів не підходять для відображення.

Тоді краще в UI не змінювати бекенд під «красивий фронтенд», а зробити перетворення на фронтенді:

```

function toListItemViewModel(dto) {
  return {
    id: dto.id,
    title: dto.title,
    subtitle: dto.status ?? "-",
  };
}

```

Це допомагає:

- не ламати контракти бекенду;
- зробити фронтенд код більш читабельним.

Типові помилки, які вирішує apiClient

- Забути перевірити `response.ok` – тепер воно в одному місці.
- Прочитати `json()` на 204 – тепер це оброблено.

- Не зрозуміло, чому Failed to fetch – тепер є єдиний текст «мережа або CORS» + деталі.
- URL бекенду розкиданий по коду – тепер є API_BASE_URL.

Стан інтерфейсу під час інтеграції

Коли фронтенд починає працювати з реальним API, з’являються задачі, яких майже не було в «статичному» UI (лабораторна робота №1):

- запит виконується не миттєво, потрібен стан «завантаження»;
- дані можуть бути порожніми – це не помилка, але це окремий сценарій;
- запит може завершитися помилкою (4xx/5xx або мережа/CORS);
- після CRUD треба оновлювати список і не плутати користувача.

Тому в лабораторній роботі №4 важливо навчитися керувати станами UI під час асинхронних операцій.

Мінімальний набір станів для кожного запиту

Для будь-якого запиту до API (завантажити список, створити запис, видалити) бажано мати 4 стани:

1. loading – запит виконується
2. success – дані отримано / дія виконана
3. empty – запит успішний, але даних немає (наприклад, порожній список)
4. error – помилка (HTTP 4xx/5xx або мережа)

Чому «empty» – окремо від «success»? Бо UX різний: «нічого не знайдено» – це нормальний кейс, а не помилка.

Як реалізувати стани без фреймворка

Без React/Angular/Vue достатньо:

- тримати «стан» у звичайному JS-об’єкті,
- мати 1–2 функції рендерингу, які малюють DOM відповідно до стану.

Приклад структури стану (дуже проста):

```
const state = {
  list: {
    status: "idle", // idle | loading | success | empty | error
    items: [],
```

```

    error: null
  }
};

```

Базові «екрани» (DOM-блоки) під стани

Практично зручно у index.html зробити місце під повідомлення та під дані:

```

<div id="listStatus"></div>
<div id="listContainer"></div>

```

Далі, в ui.js:

```

export function renderListStatus(status, error) {
  const el = document.getElementById("listStatus");

  if (status === "loading") el.innerHTML = "Завантаження...";
  else if (status === "empty") el.innerHTML = "Поки що немає записів.";
  else if (status === "error") el.innerHTML = `Помилка: ${error?.message ||
"невідома"}`;
  else el.innerHTML = "";
}

export function renderList(items) {
  const el = document.getElementById("listContainer");

  // мінімальний рендер: студенти можуть зробити таблицю, картки тощо
  el.innerHTML = "";
  for (const item of items) {
    const div = document.createElement("div");
    div.innerHTML = `${item.id} - ${item.title ?? "(без назви)"} `;
    el.appendChild(div);
  }
}

```

Приклад сценарію «завантажити список» з правильною послідовністю

У main.js:

```

import { getList } from "./apiClient.js";
import { renderListStatus, renderList } from "./ui.js";

async function loadList() {
  renderListStatus("loading");

  try {
    const items = await getList();

    if (!items || items.length === 0) {
      renderList([]); // очищаємо контейнер
      renderListStatus("empty");
      return;
    }

    renderList(items);
    renderListStatus("success");
  }
}

```

```

    } catch (err) {
      renderList([]); // прибираємо старі дані, щоб не вводити
в оману
      renderListStatus("error", err);
    }
  }
}

loadList();

```

Ключова логіка:

1. спочатку показати loading;
2. отримати items;
3. якщо порожньо – empty;
4. якщо є – success;
5. у catch – error і очистити дані, щоб не лишалися неактуальні.

UX під CRUD: що робити після створення/видалення

Після CRUD у UI є два популярні підходи:

Підхід А (простий і надійний): refetch

Після create/update/remove просто знову викликати loadList()

- Плюси:
 - завжди актуальні дані;
 - мінімум логіки на фронтенді.
- Мінуси:
 - зайвий запит (для лабораторної це нормально).

Підхід В (швидкий): локально оновити список

Додати/змінити/прибрати елемент у state.list.items і перерендерити.

- Плюси:
 - менше запитів.
- Мінуси:
 - легко зробити помилку, особливо якщо бекенд додає поля (id, createdAt).

Для цієї роботи рекомендовано Підхід А як базовий, а В – як бонус.

Відключення кнопок і «захист від подвійного кліку»

- Типова проблема:
 - користувач натискає «Створити» 3 рази – і створюється 3 записи.
- Мінімальний захист:
 - на час запиту виключати кнопку або форму.

Приклад:

```
function setFormEnabled(isEnabled) {
  document.getElementById("createBtn").disabled = !isEnabled;
}
```

У сценарії:

1. перед запитом setFormEnabled(false)
2. у finally setFormEnabled(true)

Показ помилок: що важливо для навчального UX

Помилка має бути:

- видимою (не тільки в console),
- зрозумілою (хоча б «не вдалося створити запис»),
- бажано – з деталями для налагодження (status / message).

Якщо в apiClient кидати об'єкт з status/message/details, UI може показувати коротко:

- Помилка (400): Validation error
- А в «деталях» (опційно) – details.

Для простого UI достатньо одного блоку під повідомлення.

Мінімальний шаблон «toast/повідомлення» (без бібліотек)

В index.html:

```
<div id="notice"></div>
```

В ui.js:

```
export function showNotice(text) {
  const el = document.getElementById("notice");
  el.innerHTML = text;
  // опційно: очистити через 3-5 секунд
  setTimeout(() => { el.innerHTML = ""; }, 4000);
}
```

В сценаріях CRUD:

- на успіх: `showNotice("Запис створено")`
- на помилку: `showNotice("Не вдалося створити запис")`

CORS: концептуально + практично

CORS – це ключова причина, чому «в Postman працює, а в браузері – ні». Тут важливо розділити: CORS – це не про бекенд-логіку і не про HTTP як такий, це про браузерні обмеження безпеки.

Що таке Same-Origin Policy і звідки береться CORS

Браузер застосовує політику Same-Origin Policy: сторінка, відкрита з одного origin, не може «просто так» читати відповіді з іншого origin.

Origin = схема + хост + порт, наприклад:

- Фронтенд: `http://localhost:5500`
- Бекенд: `http://localhost:3000`

Це різні origin, тому браузер за замовчуванням обмежує доступ до відповіді бекенду.

CORS (Cross-Origin Resource Sharing) – механізм, який дозволяє серверу сказати браузеру: «Окей, цьому origin можна читати мої відповіді».

Чому Postman не показує CORS-помилки

Postman / curl / .http клієнти не є браузером, вони не застосовують Same-Origin Policy. Тому:

- API може бути повністю працездатним,
- але браузер все одно заблокує доступ до відповіді, якщо CORS не налаштований.

Висновок: CORS виправляється на бекенді, а не «у fetch».

Preflight (OPTIONS): коли він з'являється

Іноді браузер перед основним запитом відправляє «перевірочний» запит OPTIONS – це називається preflight. Preflight з'являється, зокрема, коли:

- метод не простий (наприклад, PUT, PATCH, DELETE),

- або заголовки/контент «нестандартні» (типово: Content-Type: application/json для POST/PUT/PATCH).

Типова ситуація в цій роботі:

- GET працює,
- POST або DELETE раптом «ламається» – бо для них пішов preflight, а сервер не дозволив OPTIONS або заголовки.

Які заголовки CORS важливі (на рівні розуміння)

Сервер (бекенд) має повертати у відповіді заголовки, наприклад:

```
Access-Control-Allow-Origin: http://localhost:5500
```

(якому origin дозволено читати відповідь)

```
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
```

(які методи дозволено)

```
Access-Control-Allow-Headers: Content-Type, Authorization
```

(які заголовки дозволено від клієнта)

Додатково:

```
Access-Control-Allow-Credentials: true (якщо використовуються  
cookie/авторизація з credentials)
```

Важливо: для навчального проекту зазвичай достатньо «без credentials». Якщо credentials вмикаються, тоді Allow-Origin не може бути *.

Практика: налаштування CORS на Express

Нижче – рекомендований мінімум для бекенду на Express.

Варіант А: через пакет cors (рекомендовано)

Встановити:

```
npm i cors
```

У app.js / server.js (порядок важливий: CORS – до роутів):

```
import express from "express";
import cors from "cors";

const app = express();

app.use(express.json());

// 1) Явний whitelist origin (краще, ніж "*")
```

```
const allowedOrigins = [
  "http://localhost:5500",
  "http://127.0.0.1:5500",
  "http://localhost:5173",    // якщо хтось запускає через Vite
  "http://127.0.0.1:5173"
];

app.use(cors({
  origin: (origin, cb) => {
    // origin може бути undefined у деяких інструментів/запитів не з браузера
    if (!origin) return cb(null, true);
    if (allowedOrigins.includes(origin)) return cb(null, true);
    return cb(new Error("CORS: origin is not allowed"), false);
  },
  methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));

// 2) Явна відповідь на OPTIONS (preflight)
app.options("*", cors());

// ... далі маршрути app.get/app.post/...
```

Це дає:

- роботу з браузером,
- контрольовані origin,
- коректний preflight.

Варіант В: «дозволити всім» (допустимо лише як тимчасовий режим)

Як «швидко перевірити», чи проблема точно у CORS (але не як фінальна конфігурація):

```
app.use(cors());
app.options("*", cors());
```

Увага! Для справжньої задачі – обов’язково тільки whitelist; cors() без параметрів дозволяється лише на етапі діагностики.

Практичний сценарій діагностики CORS у DevTools

Якщо виникає помилки типу `TypeError: Failed to fetch` або CORS-повідомлення:

1. Відкрити DevTools → Network
2. Знайти запит до API (або OPTIONS перед ним)
3. Перевірити:
 - 3.1. Чи був OPTIONS?

3.2. Який у нього статус (200/204 – добре; 404/500 – проблема)

4. У відповіді OPTIONS чи є:

4.1. Access-Control-Allow-Origin

4.2. Access-Control-Allow-Methods

4.3. Access-Control-Allow-Headers

Найчастіші причини:

- CORS middleware підключено після роутів (не спрацьовує).
- Не обробляється OPTIONS.
- У whitelist немає реального origin (наприклад, дозволили localhost, а фронтенд на 127.0.0.1).
- Не дозволено Content-Type → preflight відхиляється.

Типові помилки студентів і як їх виправляти

1. Помилка 1: «localhost дозволено, але не працює»

- Причина: фронтенд відкритий з `http://127.0.0.1:5500`, а бекенд дозволив лише `http://localhost:5500`.
- Рішення: додати обидва варіанти у whitelist.

2. Помилка 2: GET працює, POST – ні

- Причина: POST з JSON викликав preflight OPTIONS, а він не дозволений/не оброблений.
- Рішення: `app.options("*", cors())` + дозволити Content-Type.

3. Помилка 3: CORS налаштований, але все одно «Failed to fetch»

- Причина: бекенд реально не запущений або порт інший.
- Рішення: перевірити напряму в браузері `http://localhost:3000/api/v1/...` і Network.

Мінімальний приклад повідомлення в UI

У catch на фронтенді, коли приходить помилка з `ApiClient`, можна показати:

```
function showError(err) {
  const el = document.getElementById("notice");
  el.innerHTML = `Помилка (${err.status}): ${err.message}<br>${err.details ?? ""}`;
}
```

Узгодження контрактів і «версійність без ламання»

У лабораторній роботі №4 фронтенд вперше починає реально залежати від того, що саме повертає бекенд. Навіть маленька зміна у форматі JSON може «зламати» відображення, форми або логіку обробки помилок.

Тому тут вводиться дисципліна роботи з контрактами: DTO як угода, і принцип «не ламати формат».

Контракт (DTO) як угода між фронтом і бекендом

Контракт – це домовленість про структуру даних у запитих/відповідях API:

- які поля існують,
- які типи,
- які можуть бути null,
- які обов’язкові,
- які значення допустимі.

Приклад «спискового» DTO (умовно):

```
{
  "id": "a1b2",
  "title": "Перевірка",
  "status": "Open"
}
```

Фронтенд пише код, який очікує ці поля. Якщо бекенд раптом:

- перейменує title → name,
- або видалить status,

фронтенд почне некоректно працювати (в кращому разі – порожні поля, в гіршому – помилки JS).

Що вважається «ламанням формату» (breaking change)

Для фронтенду breaking change – це будь-яка зміна, після якої старий клієнт (фронтенд) не може нормально працювати без правок.

Найтипівіші breaking changes:

- Перейменування поля
 - title → name
- Видалення поля, яке фронтенд використовує

- прибрали status
 - Зміна типу поля
 - id: number стало id: string (або навпаки)
 - Зміна вкладеності
 - було userName, стало user: { name: ... }
 - Зміна семантики без очевидної причини
 - поле date почало означати інше (наприклад, створення vs оновлення)
- Висновок: якщо формат вже використовується фронтом – його не більше не можна різко міняти.

6.3. Які зміни допустимі без ламання (backward compatible)

Щоб розвивати API без болю, використовують принципи сумісності.

Зазвичай допустимо:

- Додавати нові поля, якщо фронтенд їх не зобов'язаний читати
- Наприклад, додали priority, старий фронтенд просто ігнорує.
- Додавати поля з дефолтними значеннями (або робити їх необов'язковими).
- Розширювати перелік значень enum обережно
 - Фронтенд має вміти показувати «Unknown/–» для незнайомого значення.

Не допустимо без версійності:

- видаляти/перейменувати поля,
- міняти типи.

Мінімальна версійність у навчальному форматі: /api/v1

Для початку достатньо простого правила: усі маршрути API мають префікс:

```
GET /api/v1/...
```

Це вже дає структуру для майбутніх змін: якщо колись треба буде «ламати» контракт, це можна робити у v2, не зачіпаючи v1.

Для лабораторної не потрібно реально реалізовувати v2. Достатньо:

- використовувати v1 стабільно,

- у README коротко описати правило: «breaking changes лише з новою версією».

Практична дисципліна: «контракт як артефакт»

Щоб домовленості не жили тільки «в голові», їх корисно зафіксувати.

Найпростіший варіант: файл README або docs/contracts.md з коротким описом DTO.

Приклад мінімальної специфікації DTO (без OpenAPI)

```
ListItemDto (відповідь GET /api/v1/<entity>):
id: string - ідентифікатор
title: string - назва
status: string - стан (Open|Closed|...)
CreateDto (запит POST /api/v1/<entity>):
title: string - обов'язкове
comment?: string - необов'язкове
```

Достатньо 5–10 рядків на сутність, не більше.

Як фронтенду бути стійким до «дрібних» змін

Навіть якщо не ламати формат, все одно можливі ситуації:

- поле інколи null,
- enum отримав нове значення,
- бекенд тимчасово не віддає поле через баг.

Тому базова стійкість на фронтенді:

- показувати дефолт: `item.title ?? "(без назви)"`
- для enum: якщо значення невідоме – показати `"–"` або `"Unknown"`.

```
function renderItem(item) {
  return `
    <div class="card">
      <div><b>${item.title ?? "(без назви)}</b></div>
      <div>Статус: ${item.status ?? "–"}</div>
    </div>
  `;
}
```

Контракти помилок

В попередніх роботах вже описані узгоджені помилки API. У даній роботі важливо лише одне правило:

- фронтенд не повинен залежати від «ідеального» формату помилки,

- тому в `apiClient` робиться `fallback`: спочатку клієнт намагається прочитати JSON, але якщо це не виходить – працює з текстом.

Це дозволяє:

- не ламати UI, якщо бекенд повернув інший формат помилки,
- мати стабільний UX.

Мінімальний набір правил «не ламати формат»

1. Не перейменовувати і не видаляти поля DTO у v1.
2. Нові поля додаються як необов'язкові або з дефолтами.
3. Breaking changes можливі лише при введенні нової версії API (`/api/v2/...`).
4. Фронтенд має дефолтні значення для `null/undefined` та незнайомих `enum`.

Діагностика інтеграції в браузері (DevTools)

У цій лабораторній роботі вже є висока імовірність зіткнутися з ситуацією «не працює». Правильна реакція – не гадати, а діагностувати. Браузер дає для цього два основні інструменти:

1. DevTools → Network (що реально пішло в мережу і що прийшло у відповідь)
2. DevTools → Console (помилки JS та CORS-повідомлення)

Базова стратегія діагностики будь-якої проблеми

Коли `fetch()` «не працює»:

1. Відкрити Network і повторити дію (перезавантажити сторінку або натиснути кнопку).
2. Знайти запит до API (фільтр: Fetch/XHR).
3. Подивитися:
 - 3.1. Request URL (чи той шлях і порт)
 - 3.2. Status Code
 - 3.3. Response (тіло)
 - 3.4. Headers (особливо для CORS та Content-Type)
4. Якщо запиту немає взагалі:
 - 4.1. проблема в JS-логіці (не викликається функція, помилка до `fetch`),

4.2. або запит блокується ще до Network (підше).

5. Паралельно: Console – там часто є прямий текст помилки.

«Карта симптомів»: найтипівіші випадки і що вони означають

Випадок А: TypeError: Failed to fetch

Це означає не HTTP 404/500, а щось на рівні мережі/браузера:

- бекенд не запущений / не той порт
- CORS заблокував доступ
- preflight (OPTIONS) провалився
- неправильний протокол (http/https), або змішаний контент (підше у localhost)

Що робити:

- Network: чи є запит? чи є OPTIONS перед ним?
- Console: чи є рядок про CORS?

Випадок Б: Status 404 у Network

Це вже нормальна відповідь сервера. Типові причини:

- неправильний шлях (/api/v1/posts vs /api/posts)
- неправильна версія (v1)
- не той entity route
- id неправильний

Що робити:

- перевірити URL
- перевірити роутинг на бекенді
- відкрити цей URL напряму в браузері

Випадок В: Status 500

Бекенд впав під час обробки запиту. Що робити:

- дивитися логи бекенду (консоль сервера)
- у Response подивитися повідомлення (якщо є)
- перевірити, чи правильно сформоване тіло запиту (для POST/PUT)

Випадок Г: Unexpected end of JSON input або помилка JSON.parse

Це означає: фронтенд намагається читати JSON, але:

- тіло порожнє (часто 204 No Content)
- або прийшов не JSON (HTML-сторінка, текст)

Що робити:

- Network → Response: що реально прийшло?
- Headers → Content-Type: чи це application/json?

Випадок Д: CORS-помилка в Console (найпростіший випадок)

Браузер зазвичай пише прямо:

- який Origin заблоковано
- який заголовок відсутній
- що не дозволено в preflight

Що робити:

- на бекенді перевірити whitelist origin
- додати app.options("*", cors())
- дозволити Content-Type у allowedHeaders

Як читати Network-запит: що саме дивитися

У деталях запиту (клік по ньому):

- Headers → General
- Request URL – перевірка адреси/порту
- Request Method – GET/POST/OPTIONS
- Status Code – ключ до розуміння
- Headers → Request Headers
- Origin – звідки браузер робить запит (важливо для CORS)
- Content-Type – важливо для JSON і preflight
- Headers → Response Headers
 - Access-Control-Allow-Origin – якщо його немає, CORS може блокувати
 - Content-Type – чи це JSON

- Response
 - тіло відповіді (помилка/JSON/HTML)

Preflight у Network: як впізнати і як перевірити

Якщо перед POST/PUT/DELETE є OPTIONS запит на той самий URL – це preflight.

Для успішного preflight важливо:

- статус 204 або 200
- у Response Headers:
 - Access-Control-Allow-Origin
 - Access-Control-Allow-Methods (має містити потрібний метод)
 - Access-Control-Allow-Headers (має містити потрібні заголовки, напр. Content-Type)

Якщо OPTIONS має 404/500 – треба виправити CORS/маршрутизацію на бекенді.

Рекомендований «мінімальний лог» на фронтенді для навчальних цілей

Для навчання або відладки, корисно тимчасово логувати помилку з apiClient:

```
catch (err) {
  console.log("API error:", err); // status/message/details
  document.getElementById("notice").innerHTML =
    `Помилка (${err.status}): ${err.message}<br>${err.details ?? ""}`;
}
```

Це дає:

- видимість в UI,
- деталі в Console.

Але у продакшен такий код пускати не можна.

Типова послідовність перевірок

1. Чи запущений бекенд? (відкрити API URL у браузері)
2. Чи запущений фронтенд через http://..., а не file://?
3. Чи співпадає API_BASE_URL з реальною адресою бекенду?
4. У Network:

- 4.1. чи є запит?
- 4.2. який статус?
- 4.3. чи є OPTIONS?
- 4.4. чи є CORS?

TypeScript без фреймворка

TypeScript можна використовувати і без Angular/React/Vue. Але ключова річ: браузер не виконує TypeScript напряду – його потрібно скомпілювати в JavaScript.

Це корисно з двох причин:

1. типізація DTO зменшує кількість помилок інтеграції («очікував поле title, а його немає/інший тип»);
2. дисциплінування структури коду (apiClient, модулі, типи).

Нижче – два підходи.

Підхід А: «чистий» tsc (без бандлера, без фреймворка)

Мінімальна структура

```
frontend/
  public/
    index.html
  js/           ← сюди генерується збірка
  src/
    config.ts
    apiClient.ts
    ui.ts
    main.ts
  tsconfig.json
  package.json
```

1) Ініціалізація

У папці frontend:

```
npm init -y
npm i -D typescript
npx tsc --init
```

2) Налаштування tsconfig.json (мінімально придатне)

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
```

```

    "rootDir": "src",
    "outDir": "public/js",
    "strict": true,
    "noEmitOnError": true
  }
}

```

3) *public/index.html*

```

<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>LR4</title>
  </head>
  <body>
    <div id="notice"></div>
    <div id="listStatus"></div>
    <div id="listContainer"></div>

    <script type="module" src="/js/main.js"></script>
  </body>
</html>

```

4) *Компіляція*

одноразово:

```
npx tsc
```

або в режимі «watch»:

```
npx tsc -w
```

5) *Запуск фронтенду*

Наприклад:

```
npx http-server public -p 5500
```

Підхід B: Vite vanilla-ts (зручно, але це вже dev tooling)

Це не фреймворк – це dev server + збірка.

```

npm create vite@latest frontend -- --template vanilla-ts
cd frontend
npm i
npm run dev

```

Після цього фронтенд зазвичай на:

```
http://localhost:5173
```

Типізація DTO: що саме варто типізувати

У «vanilla TS» достатньо типізувати:

- DTO для списку/деталей (що приходить з бекенду)
- DTO для створення/оновлення (що відправляє фронтенд)
- помилку, яку кидає `ApiClient`

Приклад (заготовка)

Файл `src/dtos.ts`:

```
export type Id = string;

export interface MainEntityListItemDto {
  id: Id;
  title: string;
  status?: string; // необов'язкове поле
}

export interface CreateMainEntityDto {
  title: string;
  comment?: string;
}

export interface ApiError {
  status: number; // 0 = мережа/CORS
  message: string;
  details?: string;
  errors?: Record<string, string[]>;
}
```

TypeScript-версія `ApiClient`: мінімальна типобезпечність

Файл `src/apiClient.ts`:

```
import { API_BASE_URL } from "../config";
import type { ApiError } from "../dtos";

async function request<T>(path: string, options: RequestInit = {}): Promise<T> {
  const url = `${API_BASE_URL}${path}`;

  let response: Response;
  try {
    response = await fetch(url, options);
  } catch (e: any) {
    const err: ApiError = {
      status: 0,
      message: "Помилка мережі або CORS",
      details: e?.message ?? String(e),
    };
    throw err;
  }

  if (response.status === 204) {
    // тут T має бути void або null – для простоти повернемо як unknown
    return null as unknown as T;
  }
}
```

```

    }

    const rawText = await response.text();

    if (response.ok) {
      if (!rawText) return null as unknown as T;
      try {
        return JSON.parse(rawText) as T;
      } catch {
        // якщо API інколи повертає текст – це крайній випадок
        return rawText as unknown as T;
      }
    }
  }

  let payload: any = null;
  try { payload = rawText ? JSON.parse(rawText) : null; } catch {}

  const err: ApiError = {
    status: response.status,
    message: payload?.title ?? payload?.message ?? "HTTP помилка",
    details: payload?.detail ?? rawText ?? `HTTP ${response.status}`,
    errors: payload?.errors ?? undefined,
  };

  throw err;
}

// Далі – обгортки під ваші ендпоінти (приклад)
export async function getMainEntityList() {
  return await request<Array<any>>("/<main-entity>");
}

```

Тут застосовується:

- request<T>() з generic-типом
- ApiError
- принцип «типи DTO у окремому файлі»

Типізація в UI: мінімум користі без ускладнень

У main.ts:

- типізований список
- правильний catch (err) з приведенням типу

```

import { getMainEntityList } from "../apiClient";
import type { ApiError, MainEntityListItemDto } from "../dtos";

async function load() {
  try {
    const items = await getMainEntityList() as MainEntityListItemDto[];
    // render items
  } catch (e) {

```

```
const err = e as ApiError;
document.getElementById("notice")!.innerHTML =
  `Помилка (${err.status}): ${err.message}<br>${err.details ?? ""}`;
}
}
```

Практична частина

Загальні вимоги (для всіх рівнів)

1. Окремі процеси: фронтенд і бекенд запускаються окремо (різні порти).
2. HTTP API: взаємодія лише через API (без «прямих» доступів фронтенду до БД чи файлів).
3. Контракти (DTO): фронтенд працює з DTO, які повертає бекенд. Заборонено «підганяти» фронтенд під випадковий формат відповіді – формат має бути узгоджений і стабільний.
4. Стани інтерфейсу: для кожного запиту мають бути явно продумані стани loading / success / empty / error.
5. Помилки: обробляти як мінімум:
 - неуспішні HTTP-коди (4xx/5xx),
 - помилки мережі (бекенд недоступний),
 - помилки валідації (якщо реалізовані).
6. CORS: бекенд має бути налаштований так, щоб фронтенд у браузері міг звертатися до API без CORS-помилки (конкретні правила – у рівнях нижче).

Порядок виконання роботи

1. Ознайомитись з теоретичним матеріалом.
2. Виконати індивідуальні завдання.
3. Під'єднати фронтенд до бекенду, щоб вони працювали одночасно, фронтенд посилав записи на бекенд, бекенд їх обробляв і повертав відповідний результат.

4. Завантажити код у той же репозиторій, куди був завантажений код лабораторної роботи №1. Остаточну версію позначити тегом «0.4.0».
5. Підготувати звіт по виконаній роботі, оформлений згідно з ДСТУ 3008:2015. У звіті вказати посилання на репозиторій з п.4.
6. Також у звіті необхідно коротко описати, як запустити обидві частини, і навести приклади сценаріїв перевірки.

Вимоги за рівнями оцінювання

Виконання «на задовільно»

Реалізувати читання даних з бекенду та відображення у фронтенді.

1) Бекенд

- Додати (або підтвердити наявність) мінімум двох ендпоінтів для основної сутності:
 - GET /api/v1/<MainEntity> – список
 - GET /api/v1/<MainEntity>/{id} – деталі (або перегляд одного запису)
- Налаштувати CORS так, щоб дозволити запити з фронтенду (дозволені GET та необхідні заголовки).
 - Мінімально: дозволити origin фронтенду (наприклад, <http://localhost:5173> або інший порт).

2) Фронтенд

- Реалізувати виклики fetch() до двох ендпоінтів:
 - завантаження списку,
 - завантаження деталей.
- Вивести дані списком/таблицею (мінімум 3–5 полів DTO).
- Реалізувати стани:
 - «Завантаження...»
 - «Немає даних»
 - «Помилка завантаження» (з відображенням тексту, хоча б загального)

- Якщо відповідь не 2xx – показати повідомлення про помилку (не «мовчати»).

Результат: фронтенд у браузері показує реальні дані, що приходять з бекенду; CORS не блокує запити.

Виконання «на добре»

Реалізувати CRUD-дії (частково) + уніфіковану обробку помилок та кращий CORS.

1) Бекенд

- Додати операції для основної сутності (мінімум дві з трьох):
 - POST /api/v1/<MainEntity> – створення
 - PUT /api/v1/<MainEntity>/{id} або PATCH ... – редагування
 - DELETE /api/v1/<MainEntity>/{id} – видалення
- Реалізувати валідацію вхідних DTO (мінімум: обов’язкові поля, довжини рядків, діапазони чисел).
- Зробити централізовану обробку помилок:
 - повертати коректні HTTP-коди (400/404/409/500),
 - узгоджений формат помилки (наприклад, ProblemDetails-подібна відповідь з status, title, detail, опційно errors).
- CORS без «*»: дозволити лише конкретні origin (фронтенд), явно вказати дозволені методи (GET, POST, PUT/PATCH, DELETE) і заголовки.

2) Фронтенд

- Додати форму для створення та/або редагування (залежно від вибраних операцій).
- Реалізувати клієнтську валідацію (мінімум: required, прості обмеження) + показ помилок користувачу.
- Зробити «тонкий» шар доступу до API:
 - окремий модуль apiClient (або аналог) з функціями getList(), getById(), create(), update(), remove(),
 - єдине місце, де обробляється response.ok, парсинг JSON і помилки.

- Обробляти помилки валідації з бекенду: показати користувачу, які поля неправильні (як мінімум загальне повідомлення; краще – по полях).
Результат: користувач може не тільки читати, а й змінювати дані через UI; помилки зрозумілі; CORS налаштований акуратно.

Виконання «на відмінно»

Підвищена якість інтеграції: типізація контрактів, «не ламати формат», технічна надійність клієнта.

1) Контракти та версійність

- Впровадити просту версійність API:
 - мінімум: префікс /api/v1/... (або еквівалентний підхід, але однаково всюди).
- Продемонструвати принцип «не ламати формат»:
 - заборонено перейменовувати/видаляти поля, які вже використовує фронтенд;
 - дозволено додавати нові необов'язкові поля з дефолтами.
- У звіті коротко описати 2–3 правила сумісності DTO (людською мовою).

2) TypeScript і узгодження DTO

- Фронтенд має бути написаний на TypeScript.
- DTO мають бути типізовані (інтерфейси/типи) і використовуватись у apiClient.
- Додатковий плюс до якості (оберіть один варіант):
 - або спільний пакет/папка з типами DTO, що використовується і фронтом, і беком (якщо це організаційно можливо),
 - або OpenAPI-опис і генерація типів/клієнта (навіть часткова).

3) Надійність клієнта

- Реалізувати таймаут або скасування запиту:
 - AbortController для fetch() (наприклад, кнопка «Скасувати» або авто-таймаут 10–15 с).
- Коректно обробляти сценарії:

- бекенд вимкнений/недоступний,
- бекенд повернув 500,
- бекенд повернув 400 з помилками валідації.

4) Технічний артефакт для перевірки

- У README описати приклади запитів для перевірки.

Результат: інтеграція типобезпечна, контракти стабільні, клієнт поводить коректно в «поганих» сценаріях.

Додаткові завдання (за бонусні бали)

1. Пагінація/фільтрація/сортування на API та у фронтенді (мінімум один механізм).
2. Кешування на фронтенді (наприклад, повторне відкриття списку без зайвого запиту; інвалідовувати кеш після create/update/delete).
3. Повтор запитів (retry) лише для безпечних сценаріїв (наприклад, 429/503) з обмеженням кількості спроб.

Формальні критерії оцінювання

Задовільно

1. Frontend читає дані з бекенду (fetch) і відображає їх у UI (без ручного «забивання» JSON у код).
2. Налаштовано CORS на бекенді так, щоб запити працювали з фронтенд-оріджина (не * «наосліп», а конкретно домен фронтенду).
3. UI має мінімум три стани: **loading** / **success** / **error** (не «білий екран» при проблемі).
4. Обробляються HTTP-статуси: як мінімум 404/400 показують зрозуміле повідомлення користувачу.
5. Є базова конфігурація URL бекенду (константа/налаштування), а не «хардкод у 5 місцях».

Добре

1. Додано інтеграцію зміни даних: POST і ще мінімум один із PUT/PATCH/DELETE.
2. Є єдиний модуль API-клієнта (apiClient), через який ідуть всі запити (а не розкидані fetch по компонентах).
3. Узгоджено формат помилок із бекендом: frontend читає { code, message } і показує коректно.
4. Після мутацій UI оновлюється коректно (без дублювання/«фантомних» елементів).
5. Додано мінімальний UX: підтвердження видалення або блокування кнопок під час запиту.

Відмінно

1. Проект на **TypeScript** на фронтенді (типи DTO/відповідей).
2. Є **валідація** на фронтенді, узгоджена з бекендом (не суперечить серверній).
3. Реалізовано «**поганий сценарій**»: таймаут/мережевий збій/500 — UI не ламається і дає інструкцію, що робити.
4. Є короткий набір відтворюваних сценаріїв у README: «як перевірити GET/POST/помилку/CORS».

Контрольні запитання

1. Чому фронтенд не можна відкривати як file:///...? Які проблеми це створює для інтеграції з API?
2. Що таке origin? З яких частин він складається і чому різні порти (:5500 і :3000) означають різні origin?
3. Чим відрізняється ситуація «бекенд повернув 404/500» від ситуації «помилка мережі/CORS» з точки зору поведінки fetch()?
4. Чому fetch() зазвичай не кидає exception на 404/500? Як правильно перевіряти успішність відповіді?

5. Навіщо на фронтенді виділяти окремий модуль `apiClient`? Які типові помилки він допомагає уникнути?
6. Які дані (мінімально) корисно включати в «уніфіковану» помилку, яку `apiClient` передає в UI (наприклад, `status/message/details`)?
7. Що таке стани UI `loading` / `success` / `empty` / `error` і чому `empty` – це не те саме, що `error`?
8. Яка типова послідовність дій у коді при завантаженні списку: коли показувати `loading`, коли `empty`, коли `error`, і що робити зі «старими» даними при помилці?
9. Чому після `POST/PUT/DELETE` часто з'являється запит `OPTIONS`? Як він називається і для чого потрібен?
10. Поясніть своїми словами, що таке CORS і чому «Postman працює, а в браузері – CORS-помилка».
11. Які три ключові CORS-заголовки найчастіше потрібні для роботи браузера з API (змістовно, без дослівного заучування)?
12. Чому в CORS-конфігурації краще використовувати `whitelist origin`, а не дозволяти всім через `*` (у контексті навчальної лабораторної і реальних практик)?
13. Наведіть 2–3 приклади змін у DTO, які є `breaking change` для фронтенду (тобто «ламають формат»).
14. Які зміни в DTO зазвичай вважаються сумісними (не ламають старий фронтенд)? Наведіть 2 приклади.
15. Які кроки діагностики в DevTools треба виконати, якщо виникає `TypeError: Failed to fetch`? Що саме необхідно перевірити у вкладці Network?